

Step 2 — Writer ↔ Reviewer Loop — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic**, **Intermediate**, **Advanced**.

- [Hint 1 — The shape of the loop](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The shape of the loop

Basic Version

Two things get built here: a Reviewer (a new, small chain that reads a draft and scores it), and a loop that connects Writer and Reviewer together.

The loop's shape: Writer makes a draft. Reviewer reads it and gives a score plus feedback text. If the score isn't good enough, the Writer tries again — this time using the feedback. This repeats, but only up to a fixed number of times, so it can't go forever.

Think about what "feedback" needs to look like for the Writer to actually use it — not just a number, but text it can read and act on.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The README asks for two functions: `run_writer(notes, feedback=None) -> Draft` and `run_reviewer(draft) -> ReviewVerdict`. Notice `feedback` defaults to `None` — this one function handles *both* the first draft (no feedback yet) and every redraft (feedback from the previous round), so your prompt needs a conditional section: "if feedback is given, revise using it; otherwise, write a fresh first draft."

`ReviewVerdict` should be a structured object (Pydantic model), not a raw string — at least a `approved: bool` your loop can check with an `if`, plus `feedback: str` the Writer can read. Build it with `.with_structured_output(ReviewVerdict)` on the model, the same reliability reason Doc06 gives for preferring typed tool output over parsing free text.

The loop itself lives in `main.py` for this step (there's no Supervisor yet) — a plain Python `for` loop with a fixed `max_rounds`, breaking early if the Reviewer approves, and reporting the last draft plus "not approved" if it runs out of rounds. Missing that second exit condition is exactly the "loop forever" problem the README's own problems table warns about.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

`max_rounds` alone stops an *infinite* loop, but it doesn't stop a *wasteful* one. Picture this real case: the Reviewer rejects round 1 with feedback "add more numbers," the Writer redrafts, and the Reviewer rejects round 2 with the exact same feedback — "add more numbers" — word for word. Nothing is improving. Without extra logic, this burns every remaining round (and every remaining API call) before finally giving up at `max_rounds`, even though round 2 already proved the loop was stuck.

The real question: how do you tell "still making progress, worth another round" apart from "stuck, no point continuing"? The simplest honest signal you already have is the feedback text itself — if the Reviewer gives *identical* feedback two rounds in a row, that's a strong sign nothing changed, and it's worth stopping early rather than trusting the round counter alone.

The extra pieces:

- Keep a small history list — `[(draft, verdict), ...]` — as the loop runs, not just the latest draft. This is what makes the loop's behavior debuggable after the fact, instead of only seeing the final result.
- A check before each redraft: if `verdict.feedback == previous_feedback`, stop early and report "no progress between rounds," distinct from "ran out of rounds" — a different failure needs a different message, so whoever reads the report knows *why* it stopped.
- Think about what should happen to the loop's *return value* when it stops early for "no progress" vs. hitting `max_rounds` vs. actually getting approved — three different outcomes, and code that treats them all identically loses information a caller (later, the Supervisor) might need.

Sketch the "no progress" check yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both stop the loop the same way — a round counter and an approval flag. Advanced adds a second, independent reason to stop: recognizing when more rounds genuinely won't help, instead of only recognizing when you've run out of them. That distinction — "we gave up because it wasn't working" vs. "we gave up because we ran out of budget" — is exactly the kind of thing a real caller (like Step 5's Supervisor) needs to know, not just "approved: false."

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make a ReviewVerdict shape: approved (yes/no), feedback (text)

make run_writer(notes, feedback=None):
    same as Step 1's chain, but the prompt also includes feedback if it was given
    return the draft text

make run_reviewer(draft):
    ask the model to score the draft and explain what's wrong
    return a ReviewVerdict

loop up to N times:
    draft = run_writer(notes, feedback)
    verdict = run_reviewer(draft)
    if verdict says approved: stop, we're done
    otherwise: feedback = verdict's feedback text, try again

if we ran out of tries: report the last draft anyway, and say it wasn't approved
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
agents/writer_agent.py:
    def run_writer(notes: str, feedback: str | None = None) -> str:
        reuse Step 1's prompt, plus an optional feedback section
        invoke the chain, return the draft

agents/reviewer_agent.py:
    class ReviewVerdict(BaseModel):
        approved: bool
        feedback: str

    def run_reviewer(draft: str) -> ReviewVerdict:
        prompt the model to critique the draft against clear criteria
        use with_structured_output(ReviewVerdict)
        return the verdict

main.py:
    MAX_ROUNDS = 3
    feedback = None
    for round_num in range(MAX_ROUNDS):
        draft = run_writer(notes, feedback)
        verdict = run_reviewer(draft)
        if verdict.approved:
            break
        feedback = verdict.feedback
    report: final draft, verdict.approved, how many rounds it took
```

Test with one draft you make deliberately weak (vague, missing key facts) to confirm the loop actually improves it, and one case designed to never satisfy the Reviewer, to confirm the loop stops at `MAX_ROUNDS` instead of looping forever. Write it yourself, then compare against the [Solution](#).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's almost the "no progress" version — fill in the missing check yourself:

```
def run_writer_reviewer_loop(notes: str, max_rounds: int = 3):
    feedback = None
    previous_feedback = None
    history = []
    draft = ""
    verdict = None

    for round_num in range(max_rounds):
        draft = run_writer(notes, feedback)
        verdict = run_reviewer(draft)
        history.append((draft, verdict))

        if verdict.approved:
            return draft, verdict, "approved", history

        # your turn: if verdict.feedback == previous_feedback (and it's not
        # the first round), stop here and return a "no_progress" status
        # instead of continuing to the next round
        ...

        previous_feedback = verdict.feedback
        feedback = verdict.feedback

    return draft, verdict, "max_rounds", history
```

Fill in the no-progress check, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same core loop at 3 completeness levels — Basic and Intermediate stop only on approval or running out of rounds; Advanced adds a third stopping condition (repeated, unchanged feedback) and a `history` list, plus a status string that tells the caller *which* of the three reasons the loop actually stopped for, instead of collapsing all "not approved" outcomes into one flag.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)